A Real-time Research Project/Societal Related Project

Report on

SmartShirts: Natural-Language Product Search over a Vision-Verified Shirt Catalogue using LLM-Generated SQL

Submitted in Partial fulfillment of requirements for B.Tech II Year II Semester course

By

T BharathChandra	**24BD1A05FQ**
S Jagruth	**24BD1A05FN**
S Vijaya	**24BD1A05FH**
K PavanBalu	**24BD1A05EX**
N YagnaSai	**24BD1A05F9**

Under the guidance of

Mr./Ms. T. Venu
<Designation>, <Department>



KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH.

Narayanaguda, Hyderabad,Telangana-29

2025-26

KESHAV MEMORIAL INSTITUTE OF TECHNOLOGY
(AN AUTONOMOUS INSTITUTION)
Accredited by NBA & NAAC, Approved by AICTE, Affiliated to JNTUH
Narayanaguda, Hyderabad,Telangana-29

CERTIFICATE

This is to certify that this is a bonafide record of the project report titled "SmartShirts: Natural-Language Product Search over a Vision-Verified Shirt Catalogue using LLM-Generated SQL"which is being presented as the Real-time Research Project / Societal Related Project report by

1. **T BharathChandra <24BD1A05FQ >**
2. **S Jagruth <24BD1A05FN >**
3. **S Vijaya <24BD1A05FH >**
4. **K PavanBalu <24BD1A05EX >**
5. **N YagnaSai <24BD1A05F9 >**

In partial fulfillment for the II Year II Semester Course RTRP in KMIT affiliated to the Jawaharlal Nehru Technological University, Hyderabad

Mentors
(Ms/Mr/Dr. T. Venu

Program Coordinator
(Mr Shailesh Gangakhedkar)

Submitted for Final Project Review held on _____

Vision& Mission of KMIT

Vision

- To be the fountain head in producing highly skilled, globally competent engineers.
- Producing quality graduates trained in the latest software technologies and related tools and striving to make India a world leader in software products and services.

Mission

- To provide a learning environment that inculcates problem solving skills, professional, ethical responsibilities, lifelong learning through multi model platforms and prepares students to become successful professionals.
- To establish an industry institute Interaction to make students ready for the industry.
- To provide exposure to students on the latest hard ware and software tools.
- Topromoteresearch-basedprojects/activitiesintheemergingareasoftechnologyconvergence.
- To encourage and enable students to not merely seek jobs from the industry but also to create new enterprises.
- To induce a spirit of nationalism which will enable the student to develop, understand India's challenges and to encourage them to develop effective solutions.
- To support the faculty to accelerate their learning curve to deliver excellent service to students.

PROGRAM OUTCOMES(POs)

PO1. Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem Analysis: Identify formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences

PO3. Design/Development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct Investigations of Complex problems: Use research-based knowledge and research methods including design of experiments, analysis and inter pretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern Tool Usage: Create select, and, apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. The Engineer and Society: Apply reasoning informed by contextual knowledge to societal, health, safety. Legal und cultural issues and the consequent responsibilities relevant to professional engineering practice.

PO7. Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with societ yat large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multi disciplinary environments.

PO12. Life-Long Learning: Recognize the need for, and have the preparation and abilitytoengageinindependentandlife-longlearninginthebroadestcontextoftechnological change.



PROJECT OUTCOMES

P1: Apply data-engineering, NLP and computer-vision knowledge to build a verified, auditable 50,224-product apparel catalogue from 7.2 million raw records.
P2: Design a secure natural-language

query pipeline in which LLM-generated

SQL is AST-validated before a serverless

engine executes it.

P3: Use modern development tools -

DuckDB, SQLGlot, FastAPI, CLIP,

Hugging Face models and a hosted

NVIDIA Nemotron LLM - to deliver a

tested, one-click-runnable prototype.

P4: Communicate system behaviour and

limitations honestly through provenance

columns, labelled price estimates, test-

verified claims and documented

unsupported features.

MAPPING PROJECTOUTCOMESWITH PROGRAMOUTCOMES

PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
P1	H	H	M	M	H	L	L	M	M	M	M	M
P2	H	H	H	M	H	L	L	H	M	M	M	M
P3	M	M	M	M	H	L	L	M	M	M	M	M
P4	L	M	M	L	M	M	L	M	M	H	H	M

L– LOW M –MEDIUM H– HIGH

DECLARATION

We hereby declare that the results embodied in the dissertation entitled “SmartShirts: Natural-Language Product Search over a Vision-Verified Shirt Catalogue using LLM-Generated SQL”has been carried out by us together during the academic year 2025-26 as a partial fulfillment of the B.Tech II Year II Semester Course “Real-time Research Project / Societal Related Project”. We have not submitted this report to any other Course/College.

Student Name	Roll no.
1. T BharathChandra	<24BD1A05FQ >
2. S Jagruth	<24BD1A05FN >
3. S Vijaya	<24BD1A05FH >
4. K PavanBalu	<24BD1A05EX >

5. N YagnaSai <24BD1A05F9 >

ACKNOWLEDGEMENT

We take this opportunity to thank all the people who have rendered their full support to our project work. We render our thanks to **Dr. B L Malleswari**, Principal who encouraged us to do the Project.

We are grateful to **Mr. Neil Gogte,** Founder & Director and **Mr. S. Nitin,** Director, for facilitating all the amenities required for carrying out this project.

We express our sincere gratitude to **Ms. Deepa Ganu**, Academic Director for providing an excellent environment in the college.

We are also thankful to **Mr.Shailesh Gangakhedkar**, **Real-Time Research Project** Program Coordinator for providing us with time to make this project a success within the given schedule.

We are also thankful to our Project Mentor Mr./Ms. T. Venu, for her/his valuable guidance and encouragement given to us throughout the project work.

We would like to thank the entire KMIT faculty, who helped us directly and indirectly in the completion of the project.

We sincerely thank our friends and family for their constant motivation during the project work.

Student Name	Roll no.
T BharathChandra	<24BD1A05FQ >
S Jagruth	<24BD1A05FN >
S Vijaya	<24BD1A05FH >
K PavanBalu	<24BD1A05EX >
N YagnaSai	<24BD1A05F9 >

ABSTRACT

Online apparel search forces shoppers to translate what they see in their mind into filter widgets and keyword strings. This project builds SmartShirts, a system that answers plain-English questions about men's shirts - such as "find blue and white striped shirts under 2000 rupees" - by having a large language model write SQL, validating that SQL, and executing it against a vision-verified product catalogue of 50,224 shirts stored in a Parquet file and scanned by the DuckDB engine, all without a database server.

The catalogue itself is the second contribution: starting from 7.2 million Amazon product records, rule-based segregation, CLIP vision-model verification (shirt probability at least 0.5) and an attribute- mining pipeline produce a 44-column master in which colour, pattern, sleeve, price and fit are auditable through provenance columns. At query time the LLM's SQL is parsed into an abstract syntax tree and rejected unless it is a single SELECT of product identifiers from the one permitted table; colour predicates are normalised to exact pipe-delimited tokens so maroon is never matched as red. Results are ranked by price-provenance tier, then a Bayesian rating average (prior weight 20 reviews), then review count. A FastAPI web app serves ranked product cards, 28 automated unit tests pass, and two live end-to-end searches completed in 1.535 s and 1.29 s with correct filter constraints. The system is honestly scoped: it is a working natural-language query application, not yet an accuracy-evaluated recommender.

LISTOFFIGURES

S.No	Name of Screenshot	Page No.
1. **1.**	System architecture (query runtime and catalogue build)	___
2. **2.**	Use-case diagram of SmartShirts actors and goals	___

CONTENTS

CERTIFICATE Page ___

DESCRIPTION

PAGE

CHAPTER-1 **1**

1. INTRODUCTION **2-4**

1.1 Purpose of the project 2

1.2 Problem with Existing Systems 2

1.3 Proposed System 3

1.4 Scope of the Project 3

1.5 Architecture Diagram 4

CHAPTER–2 **5**

2. LITERATURESURVEY **6-7**

CHAPTER-3 **8**

3. SOFTWAREREQUIREMENTSPECIFICATION9-12

3.1 Introduction to SRS 9

3.2 Role of SRS 9

3.3 Requirements Specification Document 10

3.4 Functional Requirements 10

3.5 Non-Functional Requirements 11

3.6 Performance Requirements 11

3.7 Software Requirements 11

3.8 Hardware Requirements 12

CHAPTER–4 **13**

4. SYSTEMDESIGN **14-30**

4.1 Introduction to UML 14

4.2 UML Diagrams 15
4.2.1 Use Case Diagram 15
4.2.2 Sequence Diagram 18
4.2.3 State Chart Diagram 19
4.2.4 Deployment Diagram 20
4.3 TECHNOLOGIESUSED 21
CHAPTER–5 32
5. IMPLEMENTATION 33-42
5.1 Setting up connections with Raspberry PI 33
5.2 Coding the logic 35
5.3 Connecting the dashboard 40
5.4 Screenshots 42
5.5 UI Screen shots 43
CHAPTER–6 44
6. SOFTWARETESTING 45-51
6.1 Introduction 45
6.1.1 Testing Objectives 45
6.1.2 Testing Strategies 45
6.1.3 System Evaluation 48
6.1.4 Testing New System 49
6.2 Test Cases 50
CONCLUSION 52
FUTUREENHANCEMENTS 53
REFERENCES 54
BIBLIOGRAPHY 55

ACKNOWLEDGEMENT Page ___
ABSTRACT Page ___
LIST OF FIGURES Page ___
CHAPTER-1 INTRODUCTION Page ___
CHAPTER-2 LITERATURE SURVEY AND PROPOSED SYSTEM Page ___
CHAPTER-3 REQUIREMENT SPECIFICATION Page ___
CHAPTER-4 SYSTEM DESIGN Page ___
CHAPTER-5 IMPLEMENTATION Page ___
CHAPTER-6 TESTING AND RESULTS Page ___
CHAPTER-7 STATUS SUMMARY, CONCLUSION AND FUTURE SCOPE Page ___
REFERENCES Page ___

CHAPTER-1

1. INTRODUCTION

Shopping for apparel online means translating a mental picture - "a plain black full-sleeve shirt under two thousand rupees" - into a wall of filter widgets and keyword boxes. Keyword search fails on vocabulary (maroon vs red vs burgundy), filters fail on combinations, and recommendation engines built in the paper we followed require GPU pipelines that a small student team cannot operate or evaluate honestly. SmartShirts takes a different route that became practical in 2024-26: a large language model converts the shopper's sentence into SQL, a validator guarantees that SQL is safe and schema-correct, and DuckDB - an in-process analytics engine - scans a verified product catalogue stored as a single Parquet file. There is no database server, no trained ranking model to maintain and no search box to memorise.

1.1 Project Overview

The system has two halves. The offline half builds shirts_master.parquet: 50,224 men's shirts, each confirmed as a real shirt by the CLIP ViT-B/32 vision model, enriched with price, reviews, fabric, fit, sleeve, colour and pattern attributes, every attribute carrying a provenance column so its origin is auditable. The online half answers a plain-English query: a hosted NVIDIA Nemotron 3.5 Lightning model (30B parameters, 3B active) writes a SQL SELECT; SQLGlot parses it into an abstract syntax tree and rejects anything that is not a single SELECT of product identifiers from the one permitted table; DuckDB executes the surviving query against the Parquet file under a time bound; and matches are ranked and shown as product cards in a FastAPI web app. The two halves meet only at the Parquet file.

1.2 Motivation

- Keyword search cannot express combinations such as "green and white striped shirts under ₹2000"; LLM-to-SQL can, provided the generated SQL is verified before execution.
- Product catalogues are full of mis-tagged attributes - a shirt tagged "black" that is actually navy. Verifying every kept product with a vision model makes the data trustworthy before any query runs.
- Serverless analytics (DuckDB on Parquet) gives sub-second scans on a student laptop; no infrastructure, and a teammate can go from git clone to a running search with one double-click.
- The reference paper's cascaded-LLM recommender needs 1536-dimensional embedding indexes and GPU phases; this project keeps the parts that can be built and tested honestly (verified catalogue, secure text-to-SQL, transparent ranking) and defers the rest.

1.3 Problem Definition

Given a large, noisy Amazon product metadata corpus, (a) extract a trustworthy catalogue of men's shirts with auditable attributes, and (b) let an ordinary shopper query that catalogue in natural language, returning ranked, relevant products within seconds, while guaranteeing that untrusted model output can never damage the data or leak information.

1.4 Objectives

- Build and publish a vision-verified shirt catalogue (50,224 products, 44 audited columns) inside the git repository so a fresh clone runs with one click.
- Implement natural-language → LLM → SQL → validation → DuckDB → ranked cards as the core query path.
- Enforce exact colour semantics: pipe-delimited tokens keep maroon, red and burgundy distinct; "plain black" means the colour set is exactly {black} and the pattern is plain.
- Rank transparently: price-provenance tier first, then Bayesian rating average with a 20-review prior, then review count.
- Validate every generated statement at the AST level and keep API keys out of the browser - security as a hard requirement, not a guideline.
- Back all claims with automated tests (28 passing) and timed live end-to-end searches.

1.5 Scope

In scope: men's shirts/tops from the Amazon Reviews 2023 metadata; English queries; colour, pattern, sleeve, price (with labelled INR estimate at 83 INR/USD), rating and review-count attributes; browse, detail and health endpoints. Explicitly out of scope and unsupported (stated in the UI, never faked): live stock, button colour, exact size availability, personalised learning, and formal recommendation-accuracy metrics - the reviews-v2 join, sentiment blending and Recall@10/NDCG@10/MRR evaluation are the next milestone. An empty result set stays empty rather than silently relaxing the shopper's filters.

CHAPTER-2
LITERATURE SURVEY AND PROPOSED SYSTEM

2.1 Literature Survey

[1] Roy, Kumar and Kumar (IEEE Access, 2024) built a metadata- and review-based hybrid apparel recommender using cascaded large language models on the 2023 Amazon review dataset (371,902 men's shirts/tops). Their seven-phase pipeline combines TF-IDF content filtering (top-100 candidates), an item-based collaborative-filtering co-purchase graph grown from 65 million interactions (1.6 million edges), review-text concatenation over 6.62 GB of reviews, a cascaded embedding fingerprint (sentence-t5-xl then all-distilroberta-v1, 1,536 dimensions, a 2.28 GB index), Flair sentiment scoring of 1.1 million reviews, and a weighted fusion of content, embedding, sentiment and collaborative signals. Accuracy is strong, but the pipeline depends on GPU phases, gigabyte-scale artefacts and heavyweight models that a student team cannot retrain, serve or evaluate reliably.

[2] The Amazon Reviews 2023 corpus itself (BLaIR, ICLR 2024) supplies the metadata and review data used here. [3] Radford et al.'s CLIP (2021) showed that image and text encoders trained jointly can verify visually whether a product photograph matches its claimed category - the mechanism this project uses to police catalogue quality. [4] DuckDB and [5] SQLGlot provide the serverless execution and SQL-parsing layers; [6] FastAPI serves the web UI.

Surveying these works leaves one clear gap: nobody had combined (a) vision-verified catalogue quality with (b) an AST-guaranteed, prompt-free-injection text-to-SQL path and (c) a completely transparent, formula-based ranking that a reviewer can recompute - all runnable on a laptop with one double-click. That combination is the proposed system.

2.2 Existing System vs Proposed System

Aspect	**Existing: cascaded-LLM recommender [1]**	**Existing: keyword/filter search on stores**	**Proposed: SmartShirts**
Query input	Vector similarity over cascade fingerprints	Keyword box + filter widgets	Plain English, converted to SQL
Data trust	Seller metadata accepted as-is	Seller metadata accepted as-is	Every product CLIP-verified; attributes carry provenance columns
Hardware	GPU phases, 2.28 GB embedding index	Search backend (provider-side)	One Parquet file + in-process DuckDB
Ranking transparency	Weighted fusion of four signals	Opaque	Price tier, then Bayesian rating, then review count - re-computable
Safety of model output	Models never emit executable queries	No model involved	LLM emits SQL; AST validator hard-blocks everything unsafe
Setup for a teammate	Multi-script GPU pipeline	n/a	git clone, provide own API key, double-click start batch

2.3 Proposed System

The proposed architecture keeps three guarantees. First, trustworthy data: 7.2 million Amazon metadata records are reduced by rule-based segregation to 63,190 candidates, and only the 50,224 whose photographs score shirt_prob >= 0.5 under CLIP ViT-B/32 are kept; colour and pattern predictions from a Kaggle vision run are merged for all products with needs_review included but flagged, and every overwritten attribute keeps its previous value in *_original columns. Second, contained risk: the language model only ever sees the request plus the catalogue schema and colour/pattern vocabulary - never the catalogue - and its SQL must survive parsing by SQLGlot and a strict allow-list before DuckDB runs it under a time bound with external access disabled. Third, honest ranking: all matches are considered before the top-k cut; price provenance decides the tier, then a Bayesian average (prior weight 20) blends rating quality with review volume; explicit requests (cheapest, highest-rated, most reviewed) switch to their own deterministic sorts. Estimates are labelled as estimates and unsupported features are declared in the interface.

CHAPTER-3
REQUIREMENT SPECIFICATION

3.1 Functional Requirements

ID	Requirement
FR-1	Accept a plain-English shirt query from a single-page web UI and display ranked product cards.
FR-2	Convert the query to SQL by calling an OpenAI-compatible hosted LLM (NVIDIA Nemotron 3.5 Lightning by default; model, URL and key configurable via .env). Send only the request plus schema/vocabulary, never the catalogue.
FR-3	Parse returned SQL with SQLGlot and reject it unless it is exactly one SELECT of parent_asin FROM shirts - multiple statements, other tables, joins, subqueries, mutations, arbitrary functions and unknown columns are blocked.
FR-4	Normalise simple colour predicates (LIKE/contains/equality/IN) to exact pipe-delimited token matching, and map pattern aliases (striped → stripes, plaid → checked_plaid) to the catalogue vocabulary; invalid names are rejected.
FR-5	For a plain/solid request naming one colour, require the full stored colour list to equal that single colour AND pattern = plain (so "plain black" excludes navy|black and burgundy|black).
FR-6	Execute validated SQL with DuckDB on data/shirts_master.parquet under a per-search time bound; DuckDB's external-access settings are locked to the configured file.
FR-7	Rank all matches before the top-k cut: price-provenance tier, then Bayesian rating average (prior weight 20), then review count. Support deterministic alternate sorts for cheapest / highest-rated / most-reviewed requests.
FR-8	Show title, store, image, rating, review count and price per card; estimated INR prices are labelled (83 INR/USD conversion). Default results include uncertain attributes; the UI offers a toggle to exclude them.
FR-9	Expose HTTP endpoints: GET / (UI), GET /api/health, GET /api/browse, GET /api/products/{parent_asin}, POST /api/search.
FR-10	Never silently relax filters: an empty match set returns empty.
FR-11	Provide a catalogue-update script (merge_vision.py) that validates exact product-ID and image-URL agreement, backs up the previous Parquet, and preserves old values in *_original audit columns.

3.2 Non-Functional Requirements

3.2.1 Security

• API keys live only in the git-ignored local .env; the adapter never falls back to an unrelated provider's key and no key ever reaches the browser.

• Prompt-injection and malformed-output risk is contained by AST validation plus a table/column allow-list, not by trusting the model.

• DuckDB runs with external access disabled except the configured master Parquet, and its settings are locked.

3.2.2 Performance

• Two logged live end-to-end searches completed in 1.535 s and 1.29 s including LLM generation; the 50,224-row / 44-column Parquet scans in well under a second once SQL arrives.

• Every search is bounded by an execution timer so one query cannot hang the app.

3.2.3 Usability and Portability

• Single-page UI; one batch file starts the app; a fresh git clone (with the tracked master Parquet) plus the teammate's own .env key is the entire setup.

3.2.4 Reliability and Auditability

• 28 automated unit tests pass; uncertain colour/pattern predictions are kept with status/candidate audit fields; unknown values stay empty rather than guessed.

3.3 Use-Case Specification

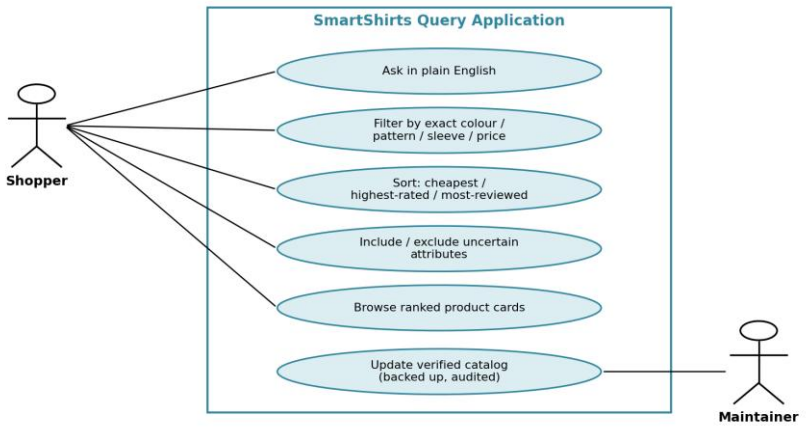


Fig. 3.1 Use-case diagram: Shopper and Maintainer goals within the SmartShirts system boundary

Use case	Actor	Description
UC-1 Search in plain English	Shopper	Type a sentence; see ranked cards within seconds.
UC-2 Apply an explicit sort	Shopper	Ask for cheapest / highest-rated / most-reviewed; a deterministic sort is used instead of blended ranking.
UC-3 Exclude uncertain attributes	Shopper	Toggle off products whose colour/pattern tags are flagged needs_review.
UC-4 Inspect a product	Shopper	Open a product's full attribute panel with provenance labels.
UC-5 Check system status	Shopper	Health endpoint reports engine readiness before searching.
UC-6 Update the catalogue	Maintainer	Run merge_vision.py to fold new predictions into shirts_master.parquet with backup and audit columns.

3.4 Hardware and Software Requirements

Category	Requirement
Hardware (run)	Windows PC, 8 GB+ RAM (16 GB tested), internet for LLM calls
Hardware (build)	GPU notebook (Kaggle) for the CLIP verification and colour/pattern vision runs
OS / runtime	Windows 11, Python 3.11+
Data engine	DuckDB (in-process, serverless)
Web stack	FastAPI + Uvicorn, single-page HTML frontend
SQL layer	SQLGlot (parser/validator)
Models & APIs	NVIDIA NIM hosted Nemotron 3.5 Lightning (configurable); CLIP ViT-B/32 (offline build)
Catalogue	shirts_master.parquet (50,224 × 44), tracked in git

CHAPTER-4
SYSTEM DESIGN
4.1 Architecture Overview

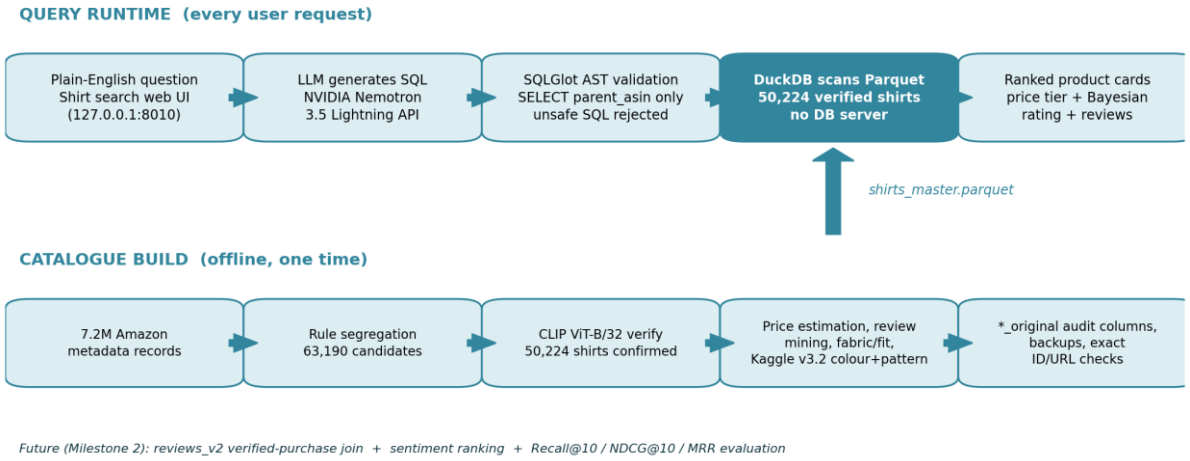


Fig. 4.1 End-to-end architecture: query runtime (top) meets the offline catalogue build (bottom) only at shirts_master.parquet

The query runtime is a strict cascade: the browser POSTs the sentence to FastAPI; the LLM adapter builds a prompt containing only the request, the catalogue schema and the colour/pattern vocabulary, and calls the hosted model; the returned JSON's SQL string goes to the validator; only SQL that passes is executed by DuckDB against the Parquet file; the ranking step reorders every match and truncates to top-k; the browser renders cards. The catalogue build is a funnel: 7.2M metadata records → 63,190 rule-segregated candidates → 50,224 CLIP-verified shirts → attribute mining (price, reviews, fabric/fit, sleeves) → Kaggle v3.2 colour/pattern merge → the audited 44-column master. Milestone-2 lines (reviews-v2 and sentiment join) plug into the ranking stage without touching the query path.

4.2 SQL Validation Rules (design of the safety gate)

- Parsed with SQLGlot into an AST; only one statement may exist.
- The statement must be SELECT parent_asin FROM shirts - no other table, no join, no subquery, no expression beyond the allow-listed columns.
- Mutations (INSERT/UPDATE/DELETE/DROP/ATTACH/COPY) and arbitrary functions are rejected before execution, not filtered after.
- Colour predicates that use LIKE, contains, equality or IN against a simple literal are rewritten by the validator to exact pipe-token matching, so the semantics cannot depend on the model's SQL style.
- Pattern values must belong to the catalogue vocabulary (plain, three stripe directions, checked_plaid, polka_dot, floral, graphic_print, camouflage, color_block, other, unknown after alias mapping); invalid names are rejected.

4.3 Data Design

shirts_master.parquet holds 50,224 rows and 44 columns in four families. Identity and commerce: parent_asin, title, store, image_url, price_usd, price_source, price_inr_est, average_rating, rating_number, review_count. Visual verdict: vision_status, shirt_prob-level signals, sleeve (with sleeve_original/sleeve_source/sleeve_evidence audit triplet), vision_reason. Mined attributes with per-field provenance: fabric + fabric_source, fit_type + fit_source + fit_review_n, pocket + pocket_source, fit_feedback + fit_feedback_n. Colour/pattern predictions with uncertainty: color, pattern, color_source, color_main, color_all, color_candidates, color_status, pattern_source, pattern_candidate, pattern_score, pattern_margin, mask_quality, palette_ambiguous_share, unmatched_share, omitted_share - plus *_original columns preserving every

overwritten value. The master is version-controlled so a clone can run with no downloads.

4.4 Ranking Design

Ranking is a total order recomputable by any reviewer. Tier 0 contains products with an exact historical price; tier 1 contains estimated prices (labelled). Within a tier, products are ordered by the Bayesian average $\bar{b} = (r \cdot n + R \cdot m) / (n + m)$, where r is the product's own average rating, n its review count, R the catalogue-wide mean rating and m the prior weight of 20 reviews; rating count breaks ties. Demands such as "cheapest" switch to an explicit price sort, keeping price estimates labelled. All matches are ranked before the top-k truncation, so the cut never hides a better-ranked product.

4.5 Interface Design

Endpoint	**Purpose**
GET /	Single-page query UI
GET /api/health	Reports engine readiness (Parquet reachable, provider configured)
GET /api/browse	Paginated catalogue browsing
GET /api/products/{parent_asin}	Full audited attributes for one product
POST /api/search	Natural-language query → validated SQL → ranked cards; returns applied filter constraints
Static /assets	Product image passthrough

CHAPTER-5
IMPLEMENTATION

5.1 Development Environment

Python 3.11 on Windows 11; dependencies pinned in requirements-query.txt (duckdb, fastapi, uvicorn, sqlglot, pandas, pyarrow, httpx, python-dotenv); VS Code and Git Bash for development; Kaggle GPU notebooks for the offline vision stages; the whole application starts with Start_Query_App.bat (http://127.0.0.1:8010).

5.2 Module Structure

Module	Responsibility
app.py	FastAPI application: UI, health, browse, product detail, search endpoints
llm_sql.py	OpenAI-compatible LLM adapter: prompt construction (schema + vocabulary only), JSON response handling per provider, SQL extraction
query_engine.py	SQLGlot AST validation and colour/pattern normalisation; DuckDB execution with time bound; Bayesian ranking and alternate sorts
request_policy.py	Uncertainty-inclusion policy and filter constraints reported back to the UI
merge_vision.py	Catalogue update: exact ID/image-URL validation, Parquet backup, *_original audit columns, CSV + summary outputs
0*_job / *_enrichment scripts	Offline attribute mining: ASIN bridge, price estimation, fabric/fit/pocket/sleeve mining, style attributes
launch.py / Start_Query_App.bat	One-click start
tests/ (4 files)	28 unit tests across merge, SQL safety, ranking, adapter and API behaviour
web/	Single-page frontend

5.3 Core Search Flow (pseudocode)

```
def search(sentence, opts):
    prompt  = system_rules + schema_and_vocab + sentence      # never the data
    reply   = llm.chat(prompt, response_format=per-provider)   # NVIDIA: prompted JSON
    sql     = parse_json(reply).sql                            # enable_thinking=false
    tree    = sqlglot.parse_one(sql)
    assert  single SELECT parent_asin FROM shirts;             # else reject
    forbid  joins, subqueries, mutations, unknown cols, extra functions
    sql     = normalize_color_tokens(tree)                     # exact pipe match
    sql     = map_pattern_aliases(tree)
    asins   = duckdb.execute(sql, on=shirts_master.parquet,
                             locked_settings, timeout=T)
    ranked  = sort(all_matches, by=[price_tier, bayesian(r,n,R,20), n])
    cards   = top_k(ranked)
    return  cards, applied_constraints                         # empty stays empty
```

5.4 Key Implementation Decisions

• NVIDIA's endpoint does not honour response-format guarantees the same way OpenAI/OpenRouter do, so the adapter uses prompted JSON plus strict local parsing for Nemotron (with enable_thinking=false so the SQL JSON arrives inside the output budget), while OpenAI-compatible providers can use native json_object mode; a settable LLM_JSON_MODE=prompt escape hatch exists for any model.

• Exact token semantics are enforced by the validator, not the prompt: even if the model writes LIKE '%black%', the rewrite makes it an exact pipe-token test, so light blue is never matched as white and gold never as yellow.

• The uncertainty toggle defaults to including needs_review predictions (as specified by the catalogue owner) and searches the actual pattern field - never the uncertain top candidate.

• Empty results stay empty: silently relaxing a shopper's filter would break the trust contract of the design.

• The .env (real API key) is git-ignored and verified as never staged; the repository ships .env.example instead.

CHAPTER-6
TESTING AND RESULTS

6.1 Automated Unit Tests (28 passing)

Run with: python -m unittest discover -s "DuckDB idea/tests" -> Ran 28 tests in 0.865 s, OK. Coverage by theme:

Theme	Representative checks
Catalogue merge integrity	identity preserved for all 50,224 IDs; missing/duplicate IDs rejected; image-URL agreement enforced; byte-for-byte backup created
Exact colour semantics	maroon distinct from red; light blue distinct from white; green-and-white requires both tokens; plain-black excludes navy|black
Pattern vocabulary	striped→stripes, plaid→checked_plaid aliases; invalid pattern names rejected
SQL safety	multi-statement input, other tables, joins, subqueries, mutations, unknown columns and arbitrary functions all rejected before execution
Ranking on the real Parquet	price tier dominates; Bayesian prior of 20 lifts well-rated high-volume products; alternate sorts honoured
LLM adapter	provider routing with fixture keys; JSON parsing; no cross-provider key fallback
FastAPI behaviour	health, browse, detail and search endpoints; empty results stay empty; constraints echoed to UI

6.2 Live End-to-End Searches (recorded in docs/live-provider-check.json)

Query (plain English)	Matches	Returned	Time	Filter check
Find blue and white striped shirts under 2000 rupees	10	5	1.535 s	passed (pipe-token CONTAINS, price bound)
Find maroon plain full sleeve shirts	287	5	1.29 s	passed (exact maroon token, plain pattern, full sleeve)

Both runs went through the real hosted Nemotron model, produced SQL that passed the validator, executed against the shipped Parquet, and returned results whose constraints were re-checked locally - HTTP 200 with filter constraints verified in both cases.

6.3 Security Verification

- Fixture tests attempt injection-shaped SQL (DROP, ATTACH, multi-statement, external table references); every one is rejected by the validator before DuckDB sees it.
- DuckDB refuses non-configured paths for external access with its settings locked, so even a hypothetical allow-list escape cannot read arbitrary files.
- .env is git-ignored; the pushed repository was grepped - no real keys in any committed file (only fixture header-construction strings).

6.4 Honest Limitations

This milestone is a working query application, not an evaluated recommender: there are no Recall@k/NDCG@k numbers yet because the ground-truth join (verified-purchase reviews) is Milestone-2 work. Exact token colour matching can omit genuinely black shirts tagged with shading colours; the tagger's colour list is the catalogue's record, and the UI states this. Prices marked estimated remain estimates; no live stock, size availability or button colour is claimed anywhere.

CHAPTER-7
STATUS SUMMARY, CONCLUSION AND FUTURE SCOPE

7.1 Milestone Status

Item	Status
Vision-verified catalogue (50,224 × 44, audited)	Done, shipped in git
Natural-language → LLM → validated SQL → DuckDB	Done, live-tested
Exact colour/pattern semantics	Done, test-covered
Transparent ranking (tiers + Bayesian prior)	Done, test-covered
Web UI, browse, detail, health endpoints	Done
28 unit tests + 2 recorded live checks	Passing
reviews-v2 verified-purchase join	Next milestone
Sentiment blend (0.5·rating + 0.5·sentiment)	Planned
Ranking evaluation (Recall@10, NDCG@10, MRR)	Planned
Personalisation / session learning	Not implemented (stated)

7.2 Conclusion

SmartShirts shows that a trustworthy apparel-search prototype can be built without a GPU pipeline: verify the data with a vision model, let an LLM write the query but never let it run unsupervised, execute on a single serverless Parquet file, and rank with a formula anyone can recompute. Every claim in this report is backed by the shipped tests or the logged live checks, and everything unsupported is declared unsupported - which is itself part of the design.

7.3 Future Scope

- Join verified-purchase reviews (reviews-v2) to the master and blend sentiment into ranking (rating×0.5 + sentiment×0.5).
- Build ground-truth sets and publish Recall@10, NDCG@10 and MRR so the system can be compared with the cascaded-LLM baseline [1].
- Extend the vocabulary-bound schema to more apparel categories and size/body-fit attributes.
- Session-aware personalisation within the same validator - the safety gate does not change when queries get smarter.

REFERENCES

[1] S. Roy, V. Kumar and V. Kumar, "Metadata and Review-Based Hybrid Apparel Recommendation System Using Cascaded Large Language Models," IEEE Access, vol. 12, 2024.

[2] Y. Hou, J. Ni, Z. He, N. Mallick, "BLaIR: Scaling Language Modeling for Context-rich 1B-token Amazon Reviews," ICLR 2024; dataset at https://amazon-reviews-2023.github.io/.

[3] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision (CLIP)," ICML 2021.

[4] DuckDB Documentation - Securing Duckdb and Parquet scanning, https://duckdb.org/docs/.

[5] SQLGlot documentation (SQL parser, AST validation, transpilation), https://sqlglot.com/.

[6] FastAPI documentation, https://fastapi.tiangolo.com/.

[7] NVIDIA NIM API reference (OpenAI-compatible chat completions, Nemotron models), https://docs.api.nvidia.com/nim/.

[8] OpenAI, "Structured Outputs" guide, https://developers.openai.com/api/docs/guides/structured-outputs.